Build a small Transformer language model *(tiny GPT from scratch)*

A) OBJECTIVES

1. Understand how an autoregressive language model is trained (next-token prediction) and how it generates text.
2. Understand self-attention and the building blocks of a decoder-only Transformer (multi-head attention, MLP, residual connections, layer normalization) implemented in PyTorch.
3. Train a small GPT on a text dataset, analyze its training curves and control the text generation (temperature, top-k).
4. Implement some of the suggested variations, as explained below (part C).

In this practical session, we'll build and train from scratch a small GPT (the same architecture family as GPT-2/GPT-3) to generate Shakespeare-like text.

The notebook follows the structure of Andrej Karpathy's "Let's build GPT" lecture: you'll start from a simple bigram baseline, assemble the full Transformer, train it and inspect what its attention heads learn. The model is tiny compared with current LLMs, but architecture and training procedure are the same.

PREVIOUS TO THE SESSION:

Read and run the notebook (TEMPLATE_FAST_gpt_from_scratch.ipynb), save a copy and rename it. Review your class Transformer materials, so that you are familiar with next-token prediction and the idea of attention. If possible, in Colab, select a GPU runtime (Runtime → Change runtime type → T4 GPU).

B) EVALUATION: Submit your notebook to Moodle **by the moodle stablished deadline.** To do this, just download the colab document as .ipynb and upload that to moodle. All your answers should be stored in the .ipynb. So please, MAKE SURE the **notebook saves the outputs of your cells**.

The following will be evaluated:

- Correctness and extent of the work. (NO NEED for great-looking generated text or very low losses! This is a toy model. We just want correctness on what you have done!)
- Correctness in the use of technical terms in the answers.
- Analysis and discussion of the results.
- Organization and cleanliness of the code.

C) DESCRIPTION OF TASKS for LAB 5:

1. Run and understand the provided notebook. Use notebook **TEMPLATE_FAST_gpt_from_scratch.ipynb**.

- 1.1. **Read** the notebook, **run** it (all cells) and keep all the outputs. If you are short on time or computing, **PRESET = "fast"** or reduce **max_iters**.

Add text cells at the end of the notebook discussing each of the following 3 questions (AROUND 10 LINES OF TEXT each question):

- 1.2. **Compare** the bigram baseline and the GPT: report the final validation loss and perplexity of both, and discuss the train/validation curves (is there over-fitting?) and the differences in the generated text.
- 1.3. **Generate** text with at least 3 different combinations of temperature and top-k, and describe how each of them affects the output.
- 1.4. **Plot the attention maps** of the first and last layer **for a prompt of your choice**. Describe at least two patterns you observe and discuss what the attention maps can and cannot tell us about the model.

2. Implement two variations. Make a copy of the notebook for each one:

- **2.1.** Implement one ablation, from the following list, for the attention module in a **new copy of the notebook**:
  - removing the position embedding,
  - removing the $1/\sqrt{d_k}$ scaling,
  - removing the residual connections,
  - changing the number of heads.

  Train the unmodified model and the ablated model with **PRESET = "fast"** and the same random seed.

  Compare validation losses; discuss why each change helps or hurts, using the concepts seen in the notebook (answer in a text cell of around 10 LINES of text)
- **2.2.** Implement one modification, from the following list, in a **new copy of the notebook** (run with PRESET = “fast”, compare and discuss the effect/differences in a text cell at the end of the notebook of around 10 LINES long)
  - Implement **top-p (nucleus) sampling** in the generate function and compare it with temperature and top-k sampling using a fixed prompt of your choice (e.g. “ROMEO:”). top-p keeps the smallest set of highest-probability candidates whose cumulative probability reaches p.
  - Replace the learned **position embeddings** with sinusoidal or rotary (RoPE) embeddings.
  - **Tokenization.** Replace the character-level tokenizer by a BPE tokenizer (e.g. **tiktoken**). Compare the vocabulary size, the sequence lengths and the loss per character (not per token) with those of the character-level model. **It’s ok if you can’t run the whole notebook with this option!!**